

KAROO ONLINE — THE AFFINITY GRAPH

Graph analytics: the relationships SQL can't see
Turning a million orders into a recommendation network

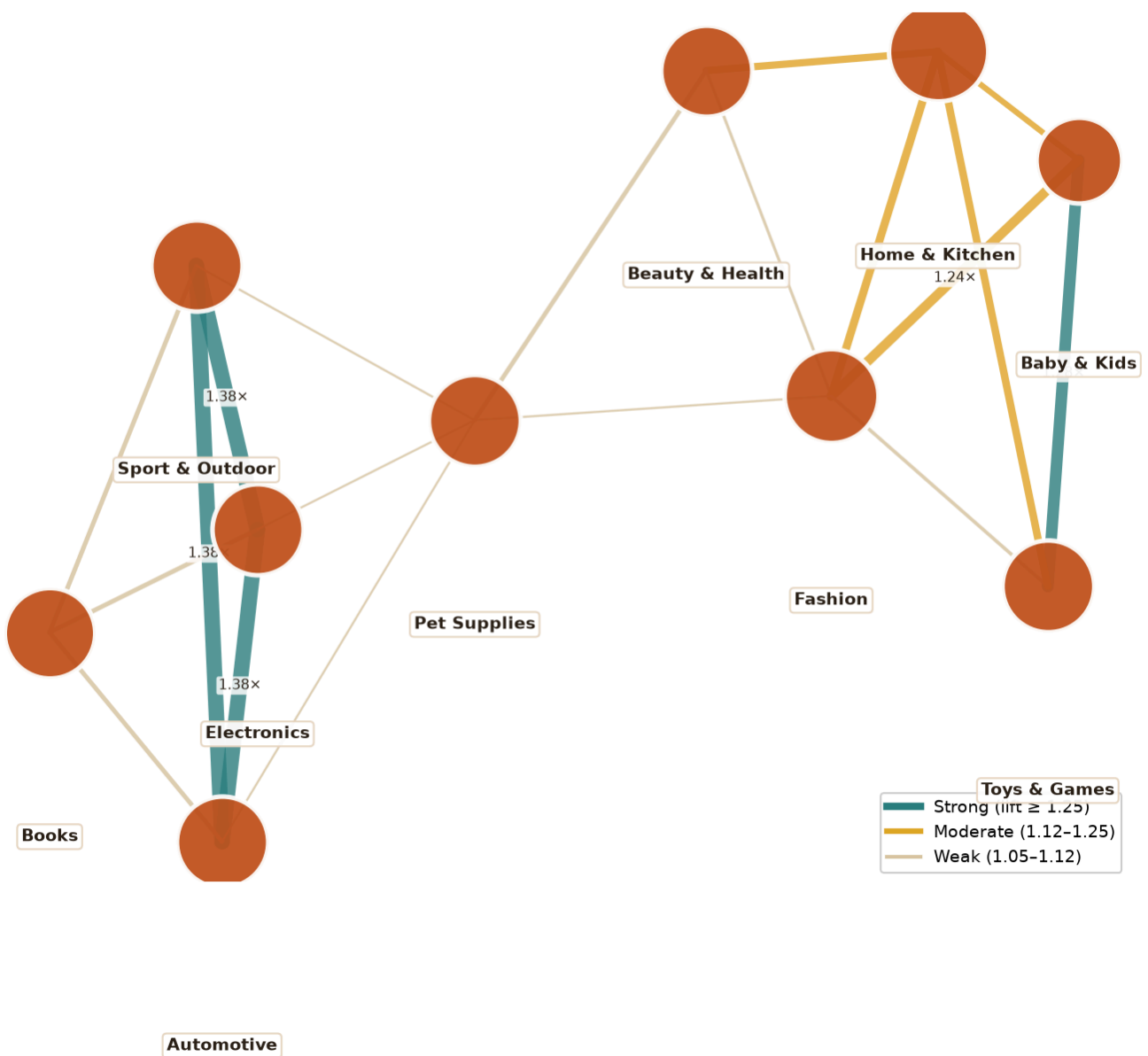
Anthony Apollis · 2026

Why a graph view?

Rows and totals answer "how much". A **graph** answers "what connects to what". By modelling customers and the categories they buy as a network, the products that genuinely travel together emerge as the strongest edges — the raw material of a recommendation engine.

The Category Affinity Network

Category Affinity Network — what's genuinely bought together
(edge thickness & label = lift; node size = co-purchase volume)



How to read it: each node is a category (bigger = more co-purchase volume); each edge is a genuine "bought-together" relationship, thicker and greener the stronger the lift.

Unlike a raw correlation, **lift controls for how popular each category already is** — so a strong edge means a real affinity, not just two common categories coinciding.

The strongest affinities

Bought together	Lift	Confidence	Customers
Automotive + Sport & Outdoor	1.38×	72%	29,989
Electronics + Sport & Outdoor	1.38×	72%	29,843
Automotive + Electronics	1.38×	72%	29,781
Baby & Kids + Toys & Games	1.28×	68%	23,240
Baby & Kids + Fashion	1.24×	70%	23,953
Fashion + Home & Kitchen	1.20×	81%	36,717
Home & Kitchen + Toys & Games	1.19×	63%	33,862
Beauty & Health + Home & Kitchen	1.18×	79%	32,893

So what — the business use: these pairs drive three concrete actions: **(1)** "customers also bought" widgets on product pages, **(2)** cart cross-sell prompts, and **(3)** bundle promotions. Because the edges are ranked by lift, the team knows exactly which pairings to surface first — the highest-ROI output of the entire analytics pipeline.

Why a graph database (Neptune) in production

At small scale this network is computed with a SQL market-basket query. At marketplace scale — millions of customers, deep multi-hop questions like "customers similar to those who bought X, via shared brand *and* province" — a dedicated graph database (Amazon Neptune) answers in a single traversal what SQL would need many escalating self-joins to express. The rule holds: **counts and totals** → **SQL**; **paths and relationships** → **graph**.